



SZOFTVERFEJLESZTÉS ÉS TESZTELÉS



Exotic Rentals

Projekt dokumentáció

Készítette

Futó Ábel

Baráth Agenor

Márton Roland

Szoftverfejlesztő és -tesztelő szak

Témavezető

Kerényi Róbert Nándor

oktató

MISKOLC, 2026

Tartalomjegyzék

1. Bevezetés	5
1.1. A projekt bemutatása és célja	5
1.1.1. Inspiráció és mintarendszerek	5
1.2. Célközönség meghatározása	6
2. Technológiák	7
2.1. Használt technológiák	7
2.1.1. Backend (.NET Core Web API)	7
2.1.2. Adatbázis (MySQL)	8
2.1.3. Frontend (React.js)	8
2.1.4. Fejlesztői eszközök	8
3. Követelményanalízis	9
3.1. Funkcionális követelmények	9
3.2. Nem funkcionális követelmények	9
3.3. Használati esetek (Use Cases)	10
4. Rendszertervezés	11
4.1. Adatbázis terv és normalizálás	11
4.1.1. ER-diagram és Adatszótár	11
4.2. Rendszerarchitektúra és komponensek	12
5. Megvalósítás	13
5.1. REST Backend fejlesztése	13
5.2. React Frontend fejlesztése	13

5.3.	WPF Desktop alkalmazás (Full Admin Panel)	14
6.	Minőségbiztosítás és Tesztelés	15
6.1.	Backend Integrációs Teszt (Swagger API)	15
6.2.	Frontend Funkcionális Tesztelés (Selenium Webdriver)	17
7.	Felhasználói útmutató	20
7.1.	Rendszerkövetelmények és Telepítés	20
7.2.	A szoftver használata	21
8.	Összegzés	22
8.1.	Projekt értékelése	22
8.2.	Továbbfejlesztési lehetőségek	22
8.3.	Biztonsági megfontolások	22
	Irodalomjegyzék	24

Ábrák jegyzéke

3.1. A rendszer használati esetei	10
4.1. Adatbázis Terv	12
4.2. A rendszer architektúrája	12
5.1. Kliens-szerver kommunikáció folyamata	13
5.2. A React alapú webalkalmazás kezdőlapja	14
5.3. A WPF asztali adminisztrációs felület	14
6.1. Swagger API dokumentáció - Általános végpontok	16
6.2. Swagger API dokumentáció - Jogosultságkezelési és foglalási végpontok	16
6.3. A végrehajtott kérés és a sikeres JSON válasz	17
6.4. Az automatizált Python teszt sikeres lefutása	19
7.1. Autóbérlés részleteinek megadása és foglalás leadása a webes felületen .	21

1. fejezet

Bevezetés

1.1. A projekt bemutatása és célja

Az Exotic egy .NET alapú webes alkalmazás, amely a luxus autóbérlést és a prémium taxis szolgáltatást egyetlen rendszerben valósítja meg. A projekt célja egy olyan modern, felhasználóbarát platform létrehozása, ahol az ügyfelek szabadon kiválaszthatják a számukra megfelelő járművet bérléshez vagy taxi szolgáltatáshoz. A rendszer célja a foglalási folyamat egyszerűsítése, az átlátható működés biztosítása, valamint egy magas színvonalú, megbízható szolgáltatás bemutatása informatikai szempontból. A projekt egyben lehetőséget ad a .NET keretrendszer, az adatbázis-kezelés és a webes alkalmazásfejlesztés gyakorlati alkalmazására is.

1.1.1. Inspiráció és mintarendszerek

A rendszer tervezésekor inspirációként és referenciaként az alábbi, prémium utazásszervező és autóbérlő platformokat vizsgáltuk meg:

- **Uber Black** (<https://www.uber.com>) / **Bolt Premium** (<https://bolt.eu>): A sofőrszolgálati modul és a valós idejű utazásszervezés logikája.
- **Sixt Luxury** (<https://www.sixt.hu>) / **Hertz Premium** (<https://www.hertz.com>): A klasszikus autóbérlési modul, szűrési feltételek és dátum alapú foglalási rendszerek struktúrája.

1.2. Célközönség meghatározása

Az Exotic azok számára készült, akik számára fontos a kényelem, a minőség és az igényes megjelenés. A szolgáltatás olyan felhasználókat céloz meg, akik a hagyományos autóbérlés és taxizás helyett luxusautóval, személyre szabott módon szeretnének közlekedni. Prémium élményt nyújt mind magánszemélyeknek, mind üzleti ügyfeleknek, akik értékelik az exkluzív, megbízható szolgáltatásokat. Különösen azok számára ideális, akik kiemelt eseményekre, üzleti utakra vagy reprezentatív megjelenéshez keresnek elegáns közlekedési megoldást.

2. fejezet

Technológiák

2.1. Használt technológiák

A rendszer architektúrája három fő pillérre épül: backend (API és üzleti logika), front-end (felhasználói felület) és adatbázis (perzisztens adattárolás). A megoldás alapelve, hogy a kliensoldali alkalmazás kizárólag a megjelenítésért és a felhasználói interakciókért felel, míg az üzleti logika és az adat-hozzáférés a szerveren történik.

2.1.1. Backend (.NET Core Web API)

A backend megvalósításához ASP.NET Core Web API-t használunk, amely magas teljesítményű és platformfüggetlen keretrendszer. Fő felelősségei:

- **Autentikáció és jogosultságkezelés:** JWT (JSON Web Token) alapú belépés, védett végpontok.
- **Üzleti logika:** Autók listázása, foglalások kezelése, státuszok frissítése.
- **Adat-hozzáférés:** Entity Framework Core használata a MySQL adatbázissal való kommunikációra.

Az alábbi példa egy tipikus végpont szerkezetét szemlélteti a projektből:

```
1 [HttpPost("/api/login")]
2 public async Task<IResult> Login(LoginDto dto, ExoticDbContext
   db)
3 {
4     var user = await db.Users.FirstOrDefaultAsync(u => u.Email
       == dto.Email);
```

```

5      if (user == null || !BCrypt.Net.BCrypt.Verify(dto.Password,
        user.Password))
6      {
7          return Results.Unauthorized();
8      }
9      // Token generalas es visszateres
10     return Results.Ok(new { message = "Login successful!", id =
        user.Id });
11 }

```

2.1.2. Adatbázis (MySQL)

Az adatok tárolására MySQL relációs adatbázist alkalmazunk. A relációs modell előnye, hogy strukturált, konzisztens adattárolást biztosít. A táblák közötti kapcsolatok idegen kulcsokkal (Foreign Keys) vannak összekötve.

2.1.3. Frontend (React.js)

A webes felhasználói felületet React alapokon valósítjuk meg, amely komponens-alapú fejlesztést tesz lehetővé. A React előnye, hogy a UI logika újrafelhasználható elemekből épül fel, és a felület gyorsan frissíthető (Single Page Application).

2.1.4. Fejlesztői eszközök

A fejlesztés során az alábbi eszközök támogatják a hatékony munkavégzést:

- **Git & GitHub:** Verziókezelés, közös kódfejlesztés.
- **Visual Studio 2022 & VS Code:** Backend és frontend fejlesztés.
- **Swagger:** API végpontok ellenőrzése és dokumentálása.

3. fejezet

Követelményanalízis

3.1. Funkcionális követelmények

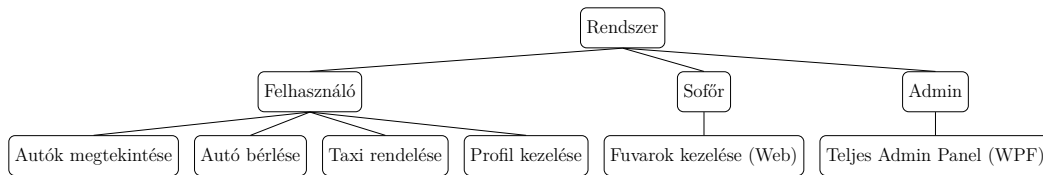
- A rendszer biztosítja a felhasználók regisztrációját és bejelentkezését jogosultságkezeléssel (felhasználó / admin).
- A felhasználók megtekinthetik az elérhető luxusautókat, azok részletes adatait (márka, típus, ár, kategória), és kiválaszthatják a kívánt járművet.
- A foglalások leadásához (2-es jogosultsági szint) a felhasználónak meg kell erősítenie az e-mail címét és fel kell töltenie a jogosítványa számát a rendszerbe.
- A rendszer lehetőséget nyújt autóbérlés és luxus taxi foglalás létrehozására, a szükséges adatok megadásával.
- A sofőrök a webes felületen, a saját profiljukban található "Sofőr Pult" alatt kezelhetik a hozzájuk rendelt taxi fuvarokat.
- Az adminisztrátorok egy teljes körű WPF asztali admin panelen keresztül kezelhetik a felhasználókat, a sofőröket, az autókat és a foglalásokat.
- A rendszer biztosítja az adatok konzisztens és biztonságos kezelését.

3.2. Nem funkcionális követelmények

- A rendszer legyen stabil és megbízható, ne omoljon össze hibás input vagy váratlan művelet esetén, és adjon érthető hibaüzenetet.
- A felület legyen átlátható és könnyen használható (UI/UX szempontok).

- A rendszer kezelje az adatokat biztonságosan: jelszavak ne kerüljenek tárolásra sima szöveggént (BCrypt hash), JWT tokenes védelem.
- Az adatbázis-műveletek legyenek konzisztensek, a módosítások azonnal tükröződjenek.

3.3. Használati esetek (Use Cases)



3.1. ábra. A rendszer használati esetei

- 1) Bejelentkezés:** A felhasználó beírja az azonosítóit, a rendszer ellenőrzi, és beengedi a megfelelő jogosultsági szintre.
- 2) Autók megtekintése:** A felhasználó megnyitja az autólistát, megnézi a részleteket.
- 3) Autóbérlés foglalás:** A hitelesített (e-mail + jogosítvány) felhasználó megadja a bérlés időtartamát, a rendszer rögzíti.
- 4) Luxus taxi foglalás és Sofőr pult:** A felhasználó leadja a taxi rendelést. A kijelölt sofőr a weboldalon a profiljába belépve elfogadja és kezeli a fuvart.
- 5) Admin funkciók:** Az admin a teljes körű WPF admin panelen kezeli a felhasználókat, a sofőröket, az autókat (rögzít, módosít, töröl) és az összes bejövő foglalást.

4. fejezet

Rendszertervezés

4.1. Adatbázis terv és normalizálás

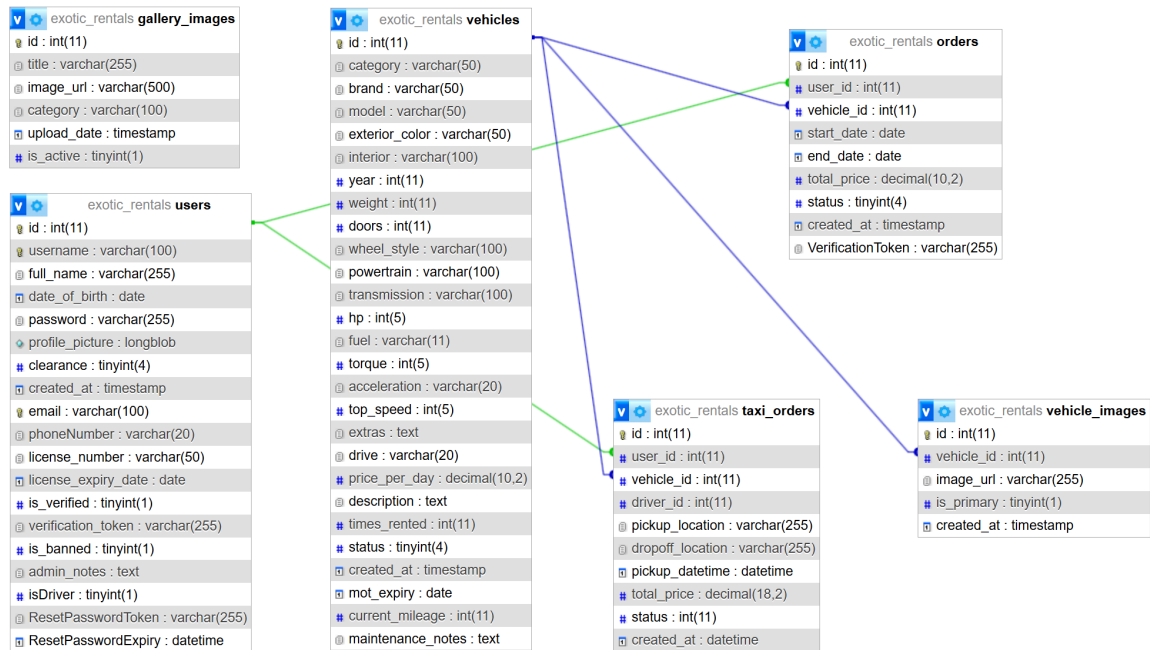
Az adatbázis tervezése során fontos szempont volt az adatok redundanciájának csökkentése és a konzisztencia biztosítása (3NF normalizációs elvek). Az egyes entitások külön táblákban kerülnek tárolásra, a kapcsolatok pedig idegen kulcsok segítségével valósulnak meg.

4.1.1. ER-diagram és Adatszótár

Az adatbázis fő entitásai:

- **Users:** a felhasználók adatai (id, email, passwordHash, clearance, licenseNumber).
- **Orders / TaxiOrders:** a foglalások adatai (dátumok, státuszok, árak).
- **Cars:** járművek törzsadatai (brand, model, specifikációk).
- **Gallery_Images:** a weboldalon megjelenő képek adatai.

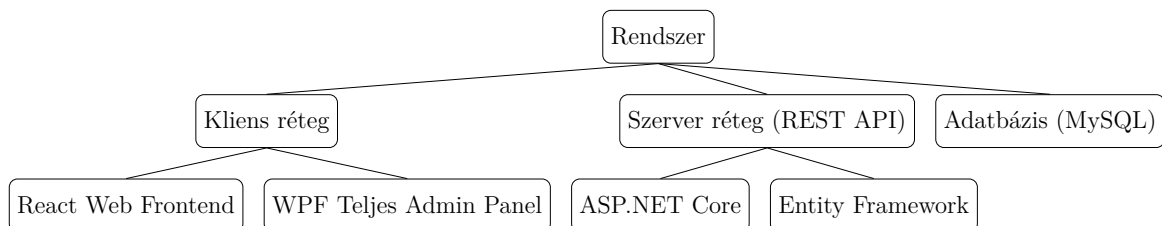
A felhasználók és a foglalások között egy-több (1:N) kapcsolat található, mivel egy felhasználó több foglalást is létrehozhat. Az alábbi ábra az alkalmazás valós relációs adatbázisának szerkezetét mutatja be:



4.1. ábra. Adatbázis Terv

4.2. Rendszerarchitektúra és komponensek

A rendszer kliens-szerver architektúrára épül. A frontend biztosítja a webes felületet a felhasználók és sofőrök számára, a backend kezeli az üzleti logikát, a WPF pedig az adminisztrátorokat szolgálja ki.



4.2. ábra. A rendszer architektúrája

5. fejezet

Megvalósítás



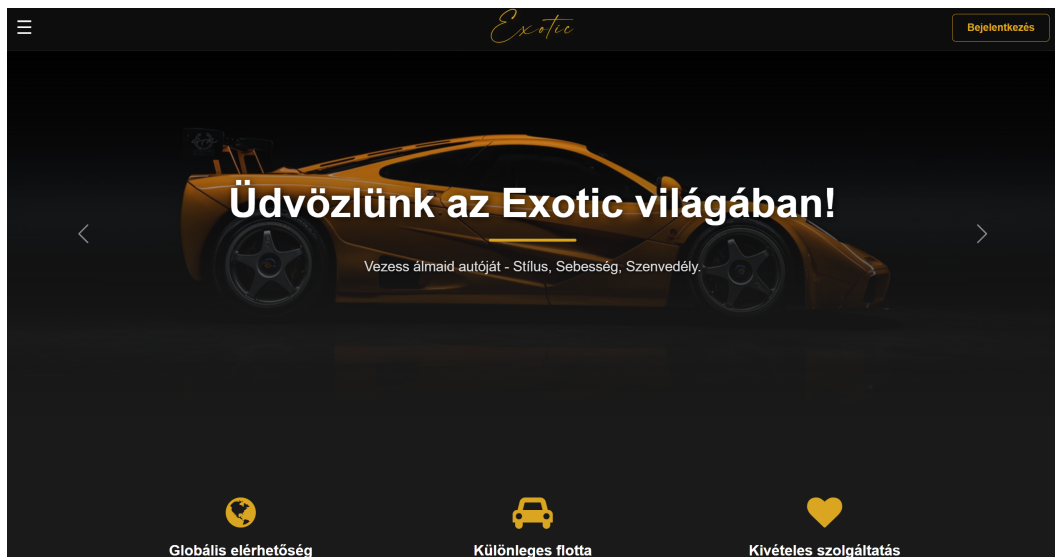
5.1. ábra. Kliens-szerver kommunikáció folyamata

5.1. REST Backend fejlesztése

A backend feladata az üzleti logika kezelése és az adatbázissal való kommunikáció. A rendszer REST API-n keresztül biztosít hozzáférést az adatokhoz a frontend számára. A rendszer a felhasználók hitelesítésére JWT (JSON Web Token) technológiát, a jelszavak védelmére BCrypt algoritmust használ.

5.2. React Frontend fejlesztése

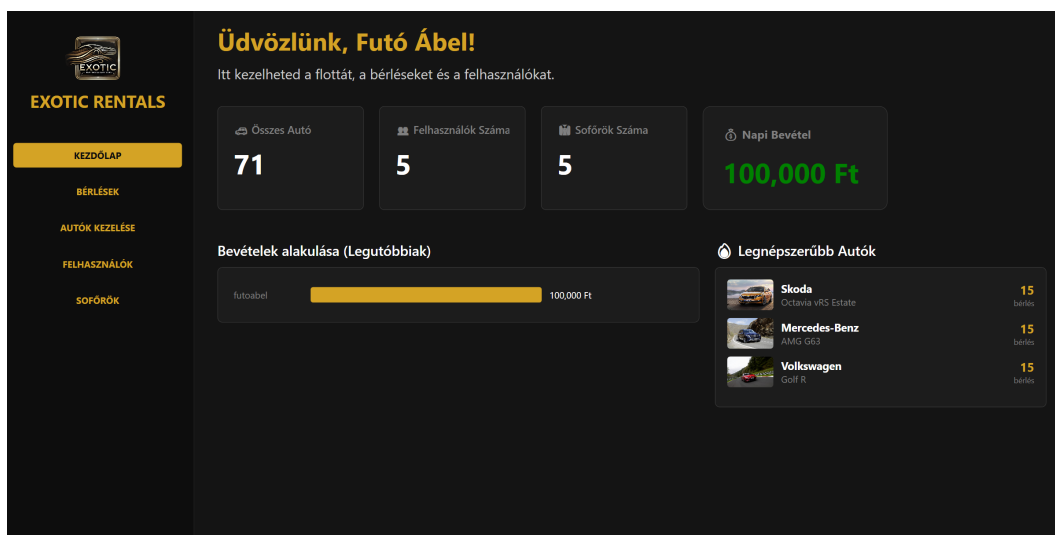
A webes felhasználói felület React keretrendszer segítségével készült. A frontend biztosítja a normál felhasználók foglалási felületét, valamint a sofőrök dedikált "Sofőr Pult" nézetét a profiljukon belül. A felület reszponzív kialakítású (CSS Flexbox/Grid), így mobiltelefonon is tökéletesen használható.



5.2. ábra. A React alapú webalkalmazás kezdőlapja

5.3. WPF Desktop alkalmazás (Full Admin Panel)

A rendszer részeként egy WPF alapú asztali alkalmazás is készült C# nyelven. Ez az alkalmazás egy teljes körű adminisztrációs panel (Full Admin Panel). Ezen a felületen az adminisztrátorok egyetlen központi helyen, adatrácsok (DataGrid) segítségével kezelhetik a felhasználókat, a sofőröket, a teljes járműflottát és a beérkezett foglalásokat. Az MVVM (Model-View-ViewModel) architektúrát használja az adatkötés (Data Binding) biztosítására.



5.3. ábra. A WPF asztali adminisztrációs felület

6. fejezet

Minőségbiztosítás és Tesztelés

A szoftver fejlesztése során a minőségbiztosítás elengedhetetlen. A tesztelés során a backend API végpontokat, valamint a frontend felhasználói felületét vizsgáltuk manuális és automatizált módszerekkel.

6.1. Backend Integrációs Teszt (Swagger API)

Az alkalmazás backendje egy RESTful API, amelyet a beépített Swagger UI segítségével teszteltünk. Ennek során a szerver és az adatbázis közötti integrációt, valamint a megfelelő JSON válaszok szerkezetét vizsgáltuk.

Tesztelt végpont: GET /api/vehicles?category=Sedan

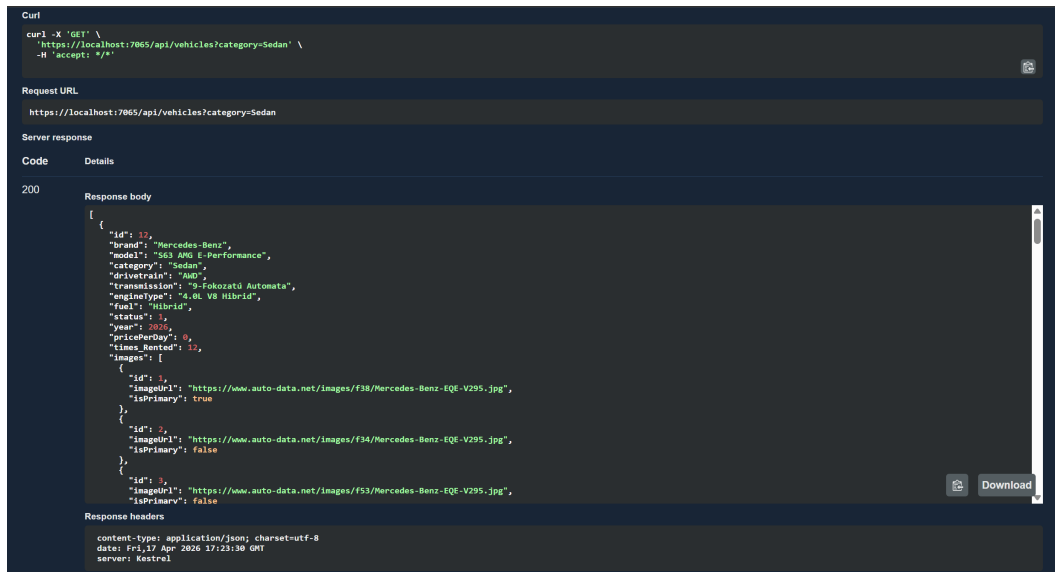
Cél: Ellenőrizni, hogy a végpont képes-e kategória alapján szűrni a járműveket, és visszaadja-e a megfelelő adatstruktúrát (benne az azonosítóval, árral, specifikációkkal és a képek listájával).

ExoticBackend			^
GET	/api/status		▼
GET	/api/vehicles		▼
GET	/api/vehicles/{id}		▼
GET	/api/gallery		▼
POST	/api/register		▼
POST	/api/auth/confirm		▼
GET	/api/auth/me/{id}		▼
GET	/api/user/{id}/profile		▼
PUT	/api/user/{id}/profile		▼
POST	/api/login		▼
GET	/api/user/{id}/orders		▼
POST	/api/orders		▼
POST	/api/orders/verify		▼

6.1. ábra. Swagger API dokumentáció - Általános végpontok

GET	/api/drivers		▼
POST	/api/orders/taxi		▼
POST	/api/orders/taxi/{id}/accept		▼
GET	/api/driver/{driverId}/taxi-orders		▼
POST	/api/orders/taxi/{id}/finish		▼
GET	/api/user/{userId}/taxi-orders		▼
POST	/api/orders/{id}/finish		▼
POST	/api/users/{id}/upload-pfp		▼
GET	/api/users/{id}/profile		▼
POST	/api/user/{id}/verify-license		▼
POST	/api/auth/forgot-password		▼
POST	/api/auth/reset-password		▼
POST	/api/user/{id}/change-password		▼

6.2. ábra. Swagger API dokumentáció - Jogosultságkezelési és foglalási végpontok



6.3. ábra. A végrehajtott kérés és a sikeres JSON válasz

Eredmény: A teszt sikeres. A szerver HTTP 200 OK státuszkóddal tért vissza, a válaszban hibátlanul megjelent a szűrésnek megfelelő jármű az összes relációs adattal együtt.

6.2. Frontend Funkcionális Tesztelés (Selenium Webdriver)

A frontend felhasználói élményének (UI/UX) és működésének ellenőrzésére automatizált funkcionális tesztek írtunk Python nyelven, a Selenium Webdriver keretrendszer használatával. A teszt a Chrome böngészőt automatizálva szimulálja egy valós felhasználó lépéseit.

Az alábbi tesztszkript ellenőrzi a sikeres bejelentkezést, majd az oldal routing logikáját (navigálást a Galéria menüpontba):

```

1 import unittest
2 from selenium import webdriver
3 from selenium.webdriver.common.by import By
4 from selenium.webdriver.support.ui import WebDriverWait
5 from selenium.webdriver.support import expected_conditions as
   EC
6 import time
7
8 class ExoticRentalsTests(unittest.TestCase):
9     @classmethod

```

```

10     def setUpClass(cls):
11         print("Bongeszo elinditasa...")
12         cls.driver = webdriver.Chrome()
13         cls.driver.maximize_window()
14         cls.BASE_URL = "http://localhost:3000"
15
16     @classmethod
17     def tearDownClass(cls):
18         time.sleep(2)
19         cls.driver.quit()
20
21     def test_1_successful_login(self):
22         driver = self.driver
23         driver.get(self.BASE_URL)
24         wait = WebDriverWait(driver, 10)
25
26         login_dropdown_btn = wait.until(EC.
27             element_to_be_clickable((By.XPATH, "//button[contains
28                 (text(), 'Bejelentkezés')]))))
29         login_dropdown_btn.click()
30
31         email_input = wait.until(EC.
32             visibility_of_element_located((By.XPATH, "//input[
33                 @type='email']"))))
34         password_input = driver.find_element(By.XPATH, "//input
35             [@type='password']")
36
37         email_input.send_keys("gamelife2222@gmail.com")
38         password_input.send_keys("admin1")
39
40         submit_btn = driver.find_element(By.XPATH, "//button[
41             @type='submit']")
42         submit_btn.click()
43
44         profile_btn = wait.until(EC.
45             visibility_of_element_located((By.XPATH, "//button[
46                 contains(text(), 'Profil')]))))
47         self.assertTrue(profile_btn.is_displayed(), "A
48             bejelentkezés nem sikerült!")
49
50     def test_2_navigation_to_gallery(self):

```

```

42     driver = self.driver
43     wait = WebDriverWait(driver, 10)
44
45     hamburger_icon = wait.until(EC.element_to_be_clickable(
46         ((By.CSS_SELECTOR, ".hamburger-icon"))
47     ))
48     hamburger_icon.click()
49
50     gallery_link = wait.until(EC.element_to_be_clickable((
51         By.XPATH, "//a[contains(text(), 'Galeria')]"))
52     ))
53     gallery_link.click()
54
55     self.assertIn("/Gallery", driver.current_url)
56     gallery_title = wait.until(EC.
57         visibility_of_element_located((By.XPATH, "//h1")))
58     self.assertEqual(gallery_title.text, "Galeria")

```

```

end/frontend/src/Tests/Test_Exotic_Frontend.py
Böngésző elindítása...
Várakozás a bejelentkezésre...
1. Teszt SIKERES: Bejelentkezve maradunk a következő teszthez!
.Navigálás a Galéria oldalra...
2. Teszt SIKERES: A navigáció és a Routing hibátlanul működik!
.Tesztetek befejeződtek. Böngésző bezárása...

-----
Ran 2 tests in 16.668s

OK

```

6.4. ábra. Az automatizált Python teszt sikeres lefutása

Eredmény: Az automatizált teszt sikeresen lefutott. A tesztelő szkript elindította a böngészőt, sikeresen bejelentkezett, majd hibátlanul navigált a Galéria menüpontba. Mindkét teszteset összesen ~16 másodperc alatt hiba nélkül lezajlott.

7. fejezet

Felhasználói útmutató

7.1. Rendszerkövetelmények és Telepítés

A rendszer használatához az alábbi szoftveres környezet szükséges:

- **Kliens oldal:** Modern böngésző (Google Chrome, Mozilla Firefox, Microsoft Edge).
- **Szerver oldal:** .NET 8 SDK futtatókörnyezet és MySQL adatbázisszerver.
- **Frontend fejlesztéshez:** Node.js és npm csomagkezelő.

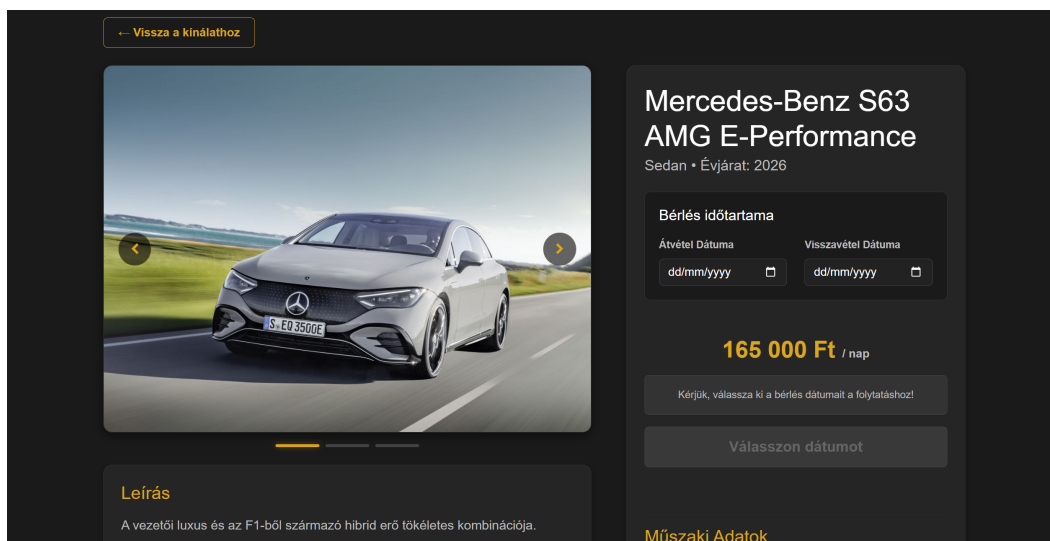
Alap telepítési és indítási folyamat (fejlesztői környezetben):

1. **Adatbázis konfigurálása:** A MySQL szerveren hozzuk létre a projekt sémáját. Az ExoticBackend mappában található `appsettings.json` fájlban állítsuk be a megfelelő *Connection String*-et.
2. **Backend indítása:** Nyissunk egy parancssort az ExoticBackend mappában. Futtassuk az Entity Framework migrációkat (`dotnet ef database update`), majd indítsuk el a szerveret a `dotnet run` paranccsal.
3. **Frontend indítása:** Lépünk be az ExoticFrontend mappába. Telepítsük a függőségeket az `npm install` paranccsal, majd indítsuk el a fejlesztői szerveret az `npm start` kiadásával.
4. **WPF kliens indítása:** A C# WPF asztali alkalmazást a Visual Studio-n vagy az exe fájlon keresztül indíthatjuk el.

7.2. A szoftver használata

A felhasználói és adminisztrációs folyamatok az alábbi fő lépésekből állnak:

1. **Böngészés:** A felhasználó megnyitja a weboldalt, ahol böngészhet a luxusautók és a szolgáltatások között.
2. **Regisztráció és Hitelesítés:** Foglalás leadásához a felhasználónak fiókot kell létrehoznia. A bérletéhez szükséges (2-es szintű) jogosultság eléréséhez a felhasználónak meg kell erősítenie az e-mail címét, és a profiljában meg kell adnia a jogosítványa számát.
3. **Bérlés vagy Foglalás:** A hitelesített felhasználó kiválasztja az autót, megadja a dátumokat (vagy a felvételi/célállomást), majd véglegesíti a rendelést.
4. **Adminisztráció és Sofőr pult:** A sofőrök a weboldalon, a profiljukba belépve a "Sofőr Pult" fül alatt tudják elfogadni és kezelni a fuvarjaikat. Az adminisztrátorok pedig a WPF alapú, teljes körű admin panelen keresztül menedzselik a felhasználókat, sofőröket, foglalásokat és autókat.



7.1. ábra. Autóbérlés részleteinek megadása és foglalás leadása a webes felületen

Gyakorlati példa egy folyamatra (Luxus Taxi szolgáltatás): Egy utas bejelentkezik a rendszerbe, megad egy felvételi címet és egy célállomást. Kiválasztja a kívánt prémium járművet, majd leadja a rendelést. A rendszer azonnal e-mail értesítést küld a kijelölt sofőrnek. A sofőr belép a weboldalra, megnyitja a "Sofőr Pult" fület, és rákattint az "Elfogad" gombra az új fuvarigénylésnél. Ekkor az utas is kap egy e-mailt arról, hogy a sofőr úton van. Az utazás végeztével a sofőr a pulton keresztül "Befejezett"-re állítja a fuvart.

8. fejezet

Összegzés

8.1. Projekt értékelése

A projekt során egy működő, komplex luxus autóbérlési és taxi szolgáltatást támogató rendszer készült. A fejlesztés során sikerült egy olyan alaparchitektúrát létrehozni, amely hatékonyan elkülöníti a felelősségi köröket (frontend, backend, asztali admin kliens), garantálva a modularitást.

8.2. Továbbfejlesztési lehetőségek

A jövőben a rendszer bővíthető például integrált online fizetési rendszerrel (pl. Stripe API), natív mobil alkalmazással (React Native alapokon), vagy a járművek GPS alapú, valós idejű nyomon követésével.

8.3. Biztonsági megfontolások

A rendszer kiemelt figyelmet fordít a felhasználói adatok védelmére. A bejelentkezés JWT alapú autentikációval történik, a jelszavakat erős hashelési eljárással tároljuk. A hitelesítési rendszer (e-mail + jogosítvány) és a backend szintű (Clearance) jogosultságellenőrzés garantálja, hogy megakadályozható legyen a jogosulatlan hozzáférés és a hibás adatok rögzítése.

Köszönetnyilvánítás

Hálásan köszönjük mindenkinek, aki szakértelmével, tanácsaival és folyamatos támogatásával segítette a vizsgaremekünk megvalósulását. Kiemelt hálával tartozunk az alábbi oktatóknak:

- **Kerényi Róbert Nándor** tanár úrnak, aki az IKT ismeretek és a backend fejlesztés terén nyújtott nélkülözhetetlen szakmai iránymutatást,
- **Négyesi Gábor** tanár úrnak a frontend programozás során nyújtott türelmes segítségéért és mentorálásáért,
- **Kasza László Róbert** tanár úrnak, akinek az adatbázisok tervezésében és az adatok hatékony kezelésében kapott precíz tanácsokat köszönhetjük,
- **Deák Csaba** tanár úrnak, aki az asztali szoftverek (WPF) fejlesztésének elsajátításában és gyakorlati alkalmazásában támogatott minket.

Végül, de nem utolsósorban köszönjük minden támogatóknak a hasznos észrevételeket és építő kritikákat, amelyekkel hozzájárultak a projektünk végleges formájának eléréséhez.

Irodalomjegyzék

- [1] React Documentation, <https://react.dev>
- [2] Microsoft .NET Documentation, <https://learn.microsoft.com/dotnet>
- [3] MySQL Documentation, <https://dev.mysql.com/doc>
- [4] Selenium Python Documentation, <https://selenium-python.readthedocs.io/>